

yassin-pellicer-project

October 7, 2026

1 Yassin Pellicer Lamla - MIADB 2026/2027

Proyecto de Python para Análisis de Datos

1.1 Inicialización de los dataFrames

```
[ ]: import pandas as pd
import numpy as np
import seaborn as sns # type: ignore
import matplotlib as plt
import matplotlib.pyplot as plt
```

<IPython.core.display.HTML object>

```
[2]: athletes_df = pd.read_csv('atletas.csv')
medals_df = pd.read_csv('medallas.csv')
```

1.2 1. Relación entre los datos de entrada

1.2.1 Análisis

1.1. Los dataFrames “atletas” y “medallas” proporcionan la información de las participaciones de cada atleta en los juegos olímpicos. Medallas muestra las participaciones, y Atletas guarda la información de cada atleta participante. Los dataFrames se relacionan con la columna ID, que referencia a cada Atleta con su respectiva participación.

```
[3]: athletes_df.info(); print()
medals_df.info(); print()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 135571 entries, 0 to 135570
Data columns (total 8 columns):
#   Column  Non-Null Count  Dtype
---  -
0   ID      135571 non-null   int64
1   Name    135571 non-null   str
2   Sex     135571 non-null   str
3   Age     129203 non-null   float64
4   Height  101655 non-null   float64
5   Weight  100686 non-null   float64
```

```

6   Team      135571 non-null  str
7   NOC       135571 non-null  str
dtypes: float64(3), int64(1), str(4)
memory usage: 8.3 MB

<class 'pandas.DataFrame'>
RangeIndex: 271116 entries, 0 to 271115
Data columns (total 8 columns):
#   Column      Non-Null Count  Dtype
---  -
0   ID          271116 non-null  int64
1   Games       271116 non-null  str
2   Year        271116 non-null  int64
3   Season      271116 non-null  str
4   City        271116 non-null  str
5   Sport       271116 non-null  str
6   Event       271116 non-null  str
7   Medal       39783 non-null   str
dtypes: int64(2), str(6)
memory usage: 16.5 MB

```

A continuación un listado con nombre del atleta, país, tipo de juegos, años en los que participó y medallas obtenidas en cada año. Para ello se deben agrupar los dos dataFrames por la columna ID. La salida estará ordenada por número de medallas en orden descendente.

```

[56]: athlete_medals_df = pd.merge(athletes_df, medals_df, on="ID")

listing = (
    athlete_medals_df
    .groupby(["ID", "Name", "NOC", "Season", "Year"], as_index=False)
    .agg(Medallas=("Medal", lambda x: list(x.dropna()))))

listing["Total_medallas"] = listing["Medallas"].apply(len)
listing = listing.sort_values("Total_medallas", ascending=False)

display(listing[["Name", "NOC", "Season", "Year", "Total_medallas"]].head(20))

```

	Name	NOC	Season	Year	Total_medallas
39484	Aleksandr Nikolayevich Dityatin	URS	Summer	1980	8
130190	Michael Fred Phelps, II	USA	Summer	2008	8
130189	Michael Fred Phelps, II	USA	Summer	2004	8
5618	Nikolay Yefimovich Andrianov	URS	Summer	1976	7
68839	Mariya Kindrativna Horokhovska	URS	Summer	1952	7
93916	Willis Augustus Lee, Jr.	USA	Summer	1920	7
157226	Mark Andrew Spitz	USA	Summer	1972	7
150714	Borys Anfiyanovych Shakhlin	URS	Summer	1960	7
15835	Matthew Nicholas "Matt" Biondi	USA	Summer	1988	7

157263	Lloyd Spencer Spooner	USA	Summer	1920	7
176280	Mikhail Yakovlevich Voronin	URS	Summer	1968	7
40915	Burton Cecil Downing	USA	Summer	1904	6
51138	Konrad Frey	GER	Summer	1936	6
130191	Michael Fred Phelps, II	USA	Summer	2012	6
130192	Michael Fred Phelps, II	USA	Summer	2016	6
124073	Kristin Otto	GDR	Summer	1988	6
25786	Vra slavsk (-Odloilov)	TCH	Summer	1968	6
178817	Hermann Otto Ludwig Weingrtnr	GER	Summer	1896	6
151066	Vitaly Venediktovich Shcherbo	EUN	Summer	1992	6
92355	Larysa Semenivna Latynina (Diriy-)	URS	Summer	1964	6

Con sólo una de las tablas (p. ej. si sólo dispusiéramos del dataframe Medals) no se podría saber nunca a quién corresponden las participaciones. Hay algunas preguntas que no se pueden responder a menos que se usen las dos tablas:

- **¿Qué atletas obtuvieron más medallas en cada año?** Necesitamos las columnas Name, NOC, y Year y Medal de la segunda
- **¿Cuál es la altura media de los atletas que ganaron una medalla de plata?** Necesitamos Height y Medal
- **¿Cuál es la proporción de condecorados entre los participantes de cada país por año?** Necesitaríamos las columnas NOC y Medals y Year

A continuación un gráfico de barras apiladas con el nº de participaciones por país para los 10 países con más participaciones. Eliminaremos los duplicados entre Atleta, Juegos y País para considerar múltiples participaciones (del mismo atleta) en el mismo año como una única participación (ya que es común que un mismo atleta participe en múltiples categorías de su deporte, como Michael Phelps - ID: 94406 en varios estilos de natación).

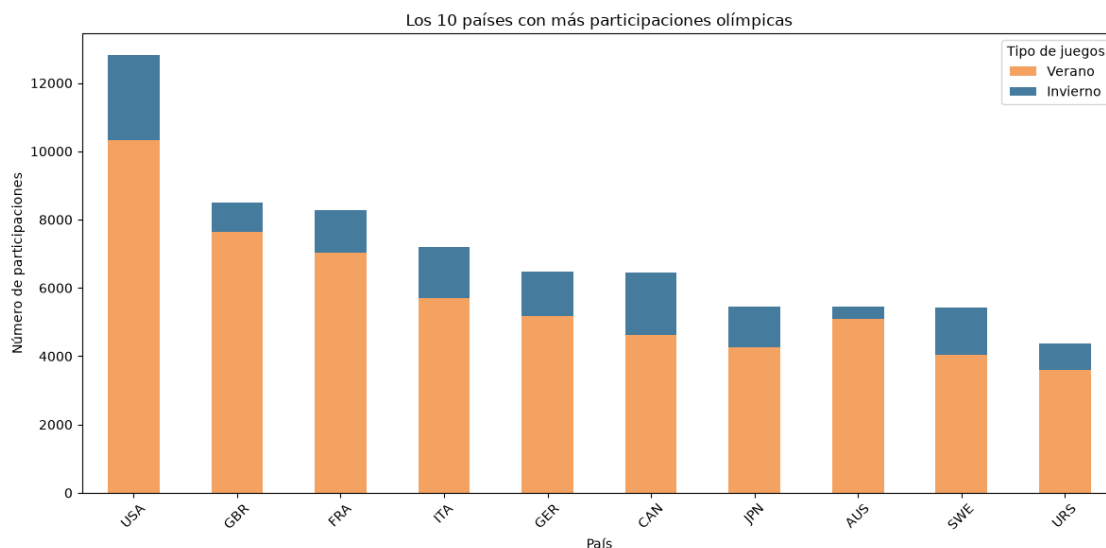
```
[5]: participaciones = (
    athlete_medals_df
    .drop_duplicates(subset=["ID", "NOC", "Games"])
    .groupby(["NOC", "Season"])
    .size()
    .unstack(fill_value=0)
)
participaciones["Total"] = participaciones.sum(axis=1)
top10 = participaciones.nlargest(10, "Total")

ax = top10[["Summer", "Winter"]].rename(
    columns={"Summer": "Verano", "Winter": "Invierno"}
).plot(
    kind="bar",
    stacked=True,
    figsize=(12, 6),
    color=["#f4a261", "#457b9d"]
)
```

```

ax.set_title("Los 10 países con más participaciones olímpicas")
ax.set_xlabel("País")
ax.set_ylabel("Número de participaciones")
ax.legend(title="Tipo de juegos")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()

```



La elección del gráfico de barras apiladas es útil porque permite ver a simple vista que las participaciones en los Juegos de Verano son mucho más abundantes que en los Juegos de Invierno.

`resumen_participacion(id_atleta, df_medallas)`

Resume los años de participación de un atleta concreto en intervalos consecutivos y devuelva un diccionario con esa información

```

[6]: def resumen_participacion(id_atleta, df):
    years = sorted(df.loc[df["ID"] == id_atleta, "Year"].unique())

    if not years:
        return {"ID": id_atleta, "participaciones": 0, "intervalos": []}

    intervals = []
    inicio = fin = int(years[0])

    for year in years[1:]:
        year = int(year)

        if year == fin + 4:
            fin = year

```

```

else:
    intervals.append(f"{inicio}-{fin}")
    inicio = fin = year

intervals.append(f"{inicio}-{fin}")
return {"ID": id_atleta, "participaciones": len(years), "intervalos":
↪intervals}

```

```
[7]: resumen_participacion(id_atleta=94406, df=medals_df)
```

```
[7]: {'ID': 94406, 'participaciones': 5, 'intervalos': ['2000-2016']}
```

1.3 2. Perfil de los atletas

1.3.1 Análisis

A continuación se obtendrán las edades medias de los atletas diferenciando entre juegos de verano e invierno. Es interesante ver que la edad de los atletas en el dataframe es la del año en el que debutan, por lo que para computar años siguientes será importante sumarle +4 en función de los años que han pasado desde su debut.

```

[8]: age_df = athlete_medals_df.copy()

# Extraemos el año de debut de cada atleta
age_df["Debut"] = age_df.groupby("ID")["Year"].transform("min")

# Calculamos para cada participación la edad del atleta
age_df["Age_of_participation"] = (
    age_df["Age"] + age_df["Year"] - age_df["Debut"]
)

# Eliminamos participaciones múltiples
age_df = age_df.drop_duplicates(subset=["ID", "Games"])

edades = (
    age_df
    .groupby("Season", as_index=False)
    .agg(Edad_media=("Age_of_participation", "mean"))
)

display(edades)

```

	Season	Edad_media
0	Summer	25.908560
1	Winter	25.102371

Ahora se mostrará un histograma de edades de los 6 países con más atletas participantes.

```

[9]: top6_paises = participaciones.nlargest(6, "Total").index

datos = age_df.loc[age_df["NOC"].isin(top6_paises)]
edades_validas = datos["Age_of_participation"].dropna()

bins = range(
    int(edades_validas.min()),
    int(edades_validas.max()) + 3,
    2
)

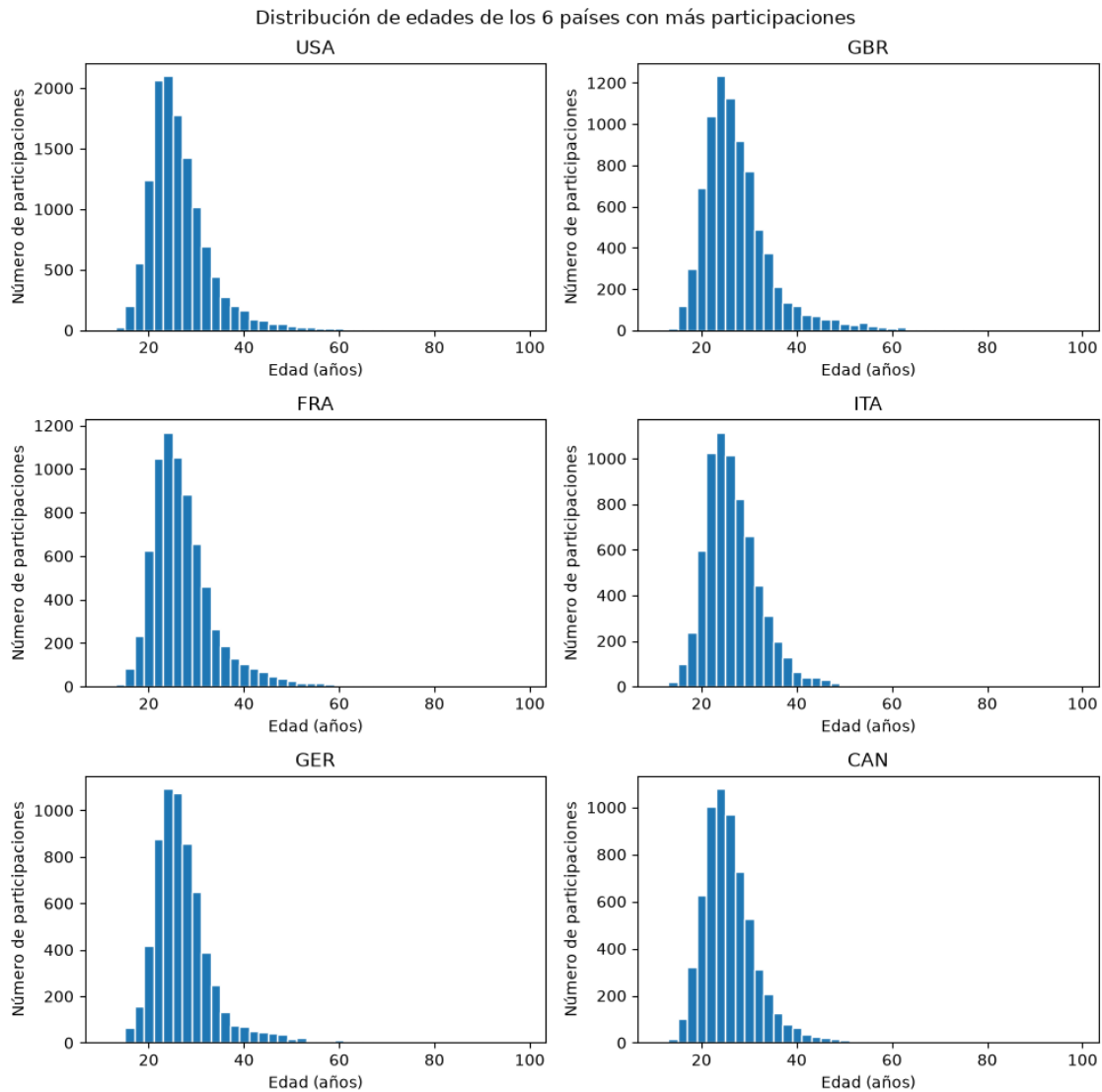
fig, axes = plt.subplots(
    3, 2, figsize=(10, 10)
)
axes = axes.ravel()

for ax, pais in zip(axes, top6_paises):
    edades_pais = datos.loc[
        datos["NOC"] == pais, "Age_of_participation"
    ].dropna()

    ax.hist(edades_pais, bins=bins, edgecolor="white")
    ax.set_title(pais)
    ax.set_xlabel("Edad (años)")
    ax.set_ylabel("Número de participaciones")

fig.suptitle(
    "Distribución de edades de los 6 países con más participaciones"
)
plt.tight_layout()
plt.show()

```



Ahora se mostrará la distribución porcentual entre hombres y mujeres y entre Juegos de Verano e Invierno

```
[10]: segregation = (
    athlete_medals_df
    .drop_duplicates(subset=["ID", "NOC", "Games"])
    .groupby(["Season", "Sex"])
    .size()
    .unstack(fill_value=0)
)

segregation["Total"] = segregation.sum(axis=1)
segregation["% Men"] = (segregation["M"] / segregation["Total"])*100
```

```

segregation["% Women"] = (segregation["F"] / segregation["Total"])*100

porcentajes = segregation[["% Men", "% Women"]].rename(
    index={"Summer": "Verano", "Winter": "Invierno"},
    columns={"% Men": "Hombres", "% Women": "Mujeres"}
)

ax = porcentajes.plot(
    kind="bar",
    stacked=True,
    figsize=(8, 5),
    color=["#457b9d", "#f4a261"],
    rot=0
)

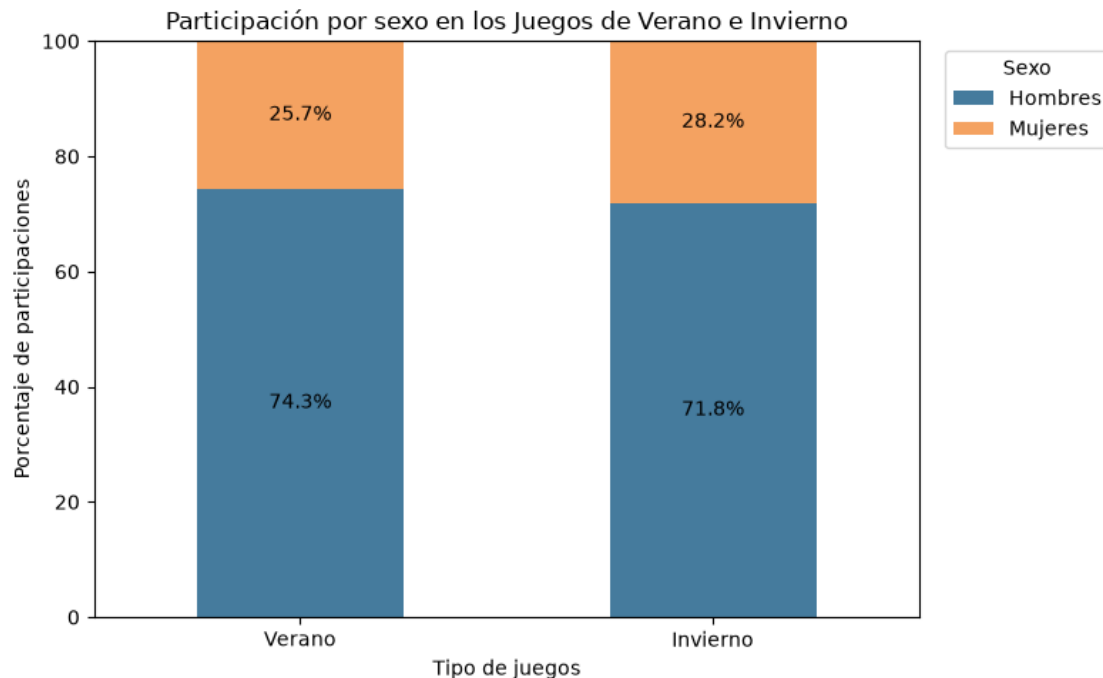
for container in ax.containers:
    ax.bar_label(container, fmt="%.1f%", label_type="center")

ax.set_title("Participación por sexo en los Juegos de Verano e Invierno")
ax.set_xlabel("Tipo de juegos")
ax.set_ylabel("Porcentaje de participaciones")
ax.set_ylim(0, 100)
ax.legend(title="Sexo", bbox_to_anchor=(1.02, 1), loc="upper left")

plt.tight_layout()
plt.show()

display(segregation)

```



Sex	F	M	Total	% Men	% Women
Season					
Summer	40787	118072	158859	74.325030	25.674970
Winter	8054	20539	28593	71.832267	28.167733

La elección del diagrama de barras apiladas es conveniente ya que permite ver cómo la suma de la proporción de hombres y mujeres conforma el 100% de participantes

Ahora se mostrará la altura y el peso medio de las mujeres en los 5 deportes en los que haya más mujeres que hombres

```
[11]: # Eliminamos los duplicados del dataframe
athlete_medals_df_nodups = athlete_medals_df.drop_duplicates(subset=["ID", "NOC", "Games"])

women_sports_df = (
    athlete_medals_df_nodups
    .groupby(["Sport", "Sex"])
    .size()
    .unstack(fill_value=0)
)

# Extraemos los 5 deportes en los que las mujeres son más numerosas
women_sports = women_sports_df[women_sports_df["F"] > women_sports_df["M"]].
    .sort_values("F", ascending=False).head(5)
```

```

# Ahora extraemos del dataFrame combinado las atletas cuyo deporte corresponde
↳ con los mencionados
datos = athlete_medals_df_nodups.loc[(athlete_medals_df["Sport"].
↳ isin(women_sports.index)) & (athlete_medals_df["Sex"] == "F")]

datos = datos[["Sport", "Weight", "Height"]]
final_df = (
    datos
    .groupby("Sport", as_index=False)
    .agg(
        Mujeres=("Sport", "size"),
        Peso_medio=("Weight", "mean"),
        Altura_media=("Height", "mean")
    )
)

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

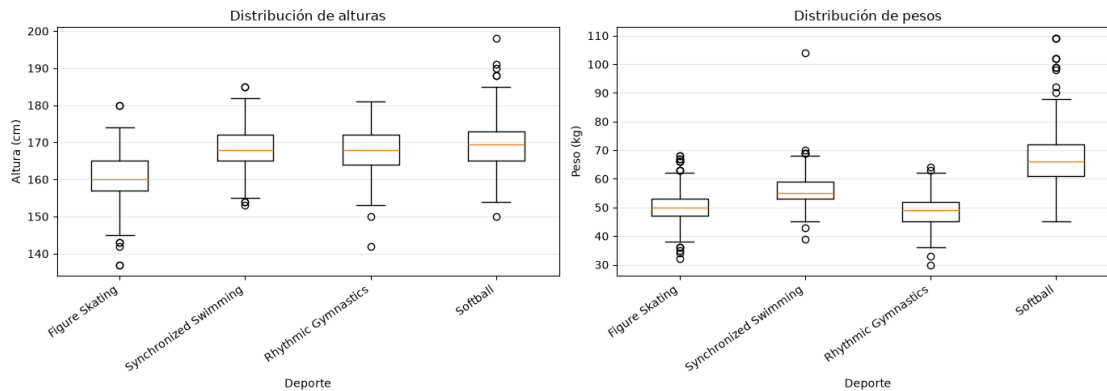
for ax, variable, titulo, unidad in [
    (axes[0], "Height", "Distribución de alturas", "Altura (cm)"),
    (axes[1], "Weight", "Distribución de pesos", "Peso (kg)")
]:
    valores = [
        datos.loc[datos["Sport"].eq(deporte), variable].dropna()
        for deporte in women_sports.index
    ]

    ax.boxplot(valores)
    ax.set_xticks(range(1, len(women_sports.index) + 1))
    ax.set_xticklabels(women_sports.index, rotation=35, ha="right")
    ax.set_title(titulo)
    ax.set_xlabel("Deporte")
    ax.set_ylabel(unidad)
    ax.grid(axis="y", alpha=0.3)

plt.tight_layout()
plt.show()

display(women_sports)
display(final_df)

```



Sex	F	M
Sport		
Figure Skating	1116	1066
Synchronized Swimming	728	0
Rhythmic Gymnastics	658	0
Softball	478	0

	Sport	Mujeres	Peso_medio	Altura_media
0	Figure Skating	1116	49.883941	160.636476
1	Rhythmic Gymnastics	658	48.760976	167.870253
2	Softball	478	67.471655	169.395089
3	Synchronized Swimming	728	55.795918	168.395415

De los datos extraídos vemos que en realidad hay menos de **5 deportes donde las mujeres compiten en mayor cantidad que los hombres**, y no 5 como se proponía inicialmente. Además, deportes como la Gimnasia Rítmica, el Softball y la Natación Sincronizada son **practicados exclusivamente por mujeres**.

`extremos_atletas(df_atletas)`

Devuelve un diccionario con los/las atletas “extremos” (más joven, más mayor, de mayor/menor altitud, y de mayor/menor peso), aplicando ciertas reglas de desempate. - Debe descartar registros sin valor numérico válido, es decir edades, alturas o pesos nulos o no plausibles (“outliers”). - Cuando haya empates, desempatar por más participaciones olímpicas (si hay información), y si persiste, por orden alfabético de nombre.

Para definir qué valores dejar fuera como outliers se hará un plot de las columnas para poder decidir

```
[12]: variables = {
    "Age": "Edad (años)",
    "Height": "Altura (cm)",
    "Weight": "Peso (kg)",
}

fig, axes = plt.subplots(3, 2, figsize=(13, 12))
```

```

for fila, (columna, etiqueta) in enumerate(variables.items()):
    grupos = [
        athletes_df.loc[athletes_df["Sex"].eq(sexo), columna].dropna()
        for sexo in ["F", "M"]
    ]

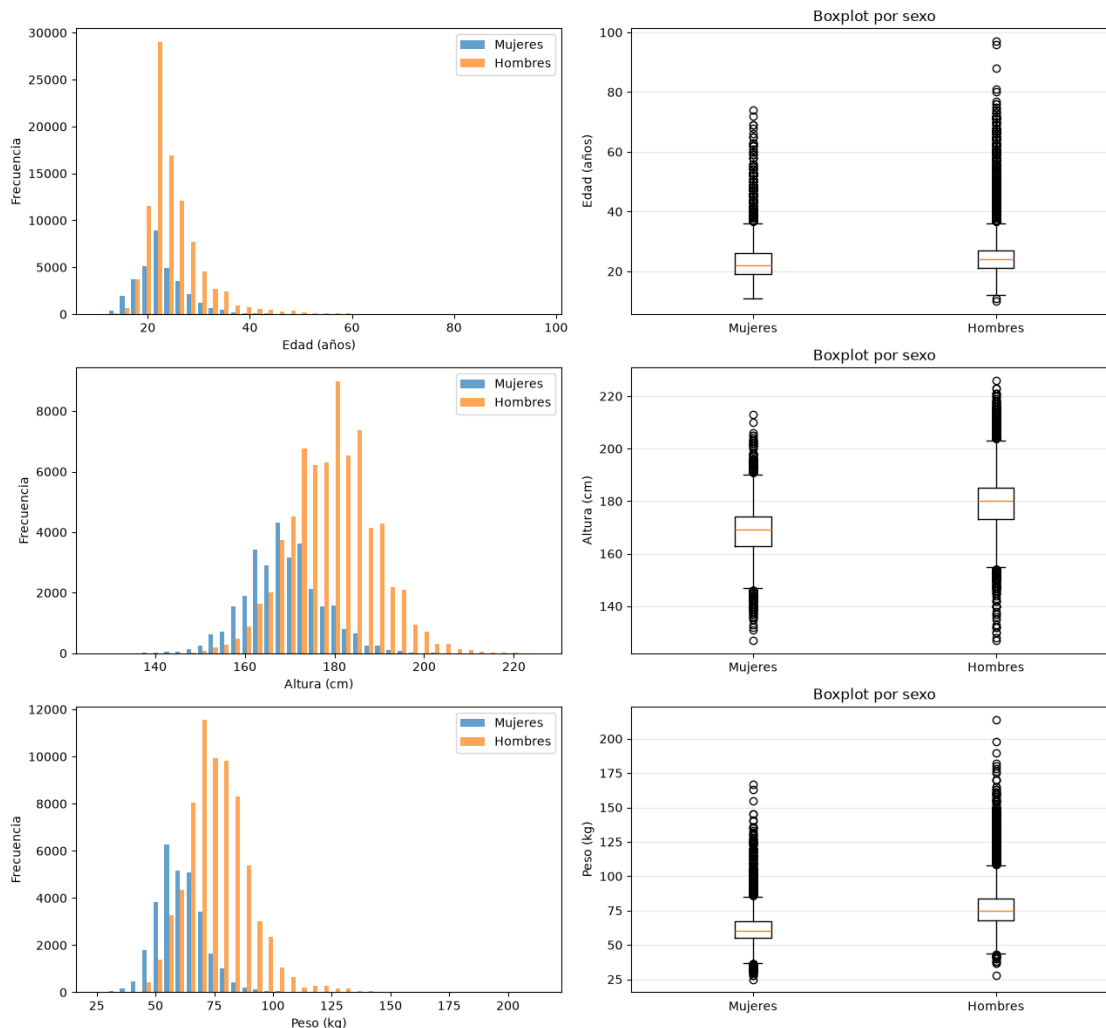
    ax_hist, ax_box = axes[fila]

    ax_hist.hist(
        grupos,
        bins=40,
        label=["Mujeres", "Hombres"],
        alpha=0.7
    )
    ax_hist.set_xlabel(etiqueta)
    ax_hist.set_ylabel("Frecuencia")
    ax_hist.legend()

    ax_box.boxplot(grupos)
    ax_box.set_xticks([1, 2])
    ax_box.set_xticklabels(["Mujeres", "Hombres"])
    ax_box.set_ylabel(etiqueta)
    ax_box.set_title("Boxplot por sexo")
    ax_box.grid(axis="y", alpha=0.3)

plt.tight_layout()
plt.show()

```



Por los histogramas y de los diagramas de caja y bigotes para cada característica puede verse que hay un gran número de valores atípicos en todas las variables analizadas. Tras estudiar el dataset y sus valores más extremos estableceremos los siguientes umbrales: - La edad máxima a la que se establecerá la participación será a los **74 años**. Esto es debido a que en la historia de los juegos olímpicos nadie ha rebasado esta edad. Todos aquellos participantes con edades superiores son personalidades históricas cuyas obras o acciones han sido parte de algunos juegos, como es el caso de John Quincy Adams Ward, un escultor americano que participó póstumamente en la Competencia de Artes de los juegos del 1910.

- La edad mínima **será de 10 años**, ya que pertenece a la del participante más joven en la historia de los juegos.

El resto de medidas, aunque extremas, corresponden todas a valores reales de deportistas que han competido en los JJ.OO

```

[13]: def extremos_atletas(df):
    criterios = {
        "mas_joven": ("Age", "min", "edad"),
        "mas_mayor": ("Age", "max", "edad"),
        "mas_alto": ("Height", "max", "altura"),
        "mas_bajo": ("Height", "min", "altura"),
        "mas_ligero": ("Weight", "min", "peso"),
        "mas_pesado": ("Weight", "max", "peso"),
    }
    resultados = {}

    for tag, (columna, extremo, key) in criterios.items():

        # Descartamos N/A - NaN
        valores = df[columna].dropna()

        # Descartamos outliers
        if columna == "Age":
            valores = valores[valores.between(10, 74)]

        # Descartamos valores vacíos
        if valores.empty:
            resultados[tag] = None
            continue

        # Extracción index
        valor_extremo = (
            valores.min()
            if extremo == "min"
            else valores.max()
        )

        indices = valores.index[valores.eq(valor_extremo)]
        empatados = df.loc[indices]

        empatados["participaciones"] = empatados["ID"].map(
            lambda id_atleta: resumen_participacion(
                id_atleta=id_atleta,
                df=medals_df
            )["participaciones"]
        )

        # Más participaciones primero; después, nombre alfabético.
        elegido = empatados.sort_values(
            ["participaciones", "Name"],
            ascending=[False, True]
        ).iloc[0]

```

```

resumen_participacion(id_atleta=94406, df=medals_df)

#Inserción Diccionario
resultados[tag] = {
    "id": int(elegido["ID"]),
    "nombre": elegido["Name"],
    key: float(elegido[columna])
}

return resultados

extremos_atletas(athletes_df)

```

```

[13]: {'mas_joven': {'id': 71691, 'nombre': 'Dimitrios Loundras', 'edad': 10.0},
      'mas_mayor': {'id': 93535, 'nombre': 'Auguste Perret', 'edad': 74.0},
      'mas_alto': {'id': 132627, 'nombre': 'Yao Ming', 'altura': 226.0},
      'mas_bajo': {'id': 82769, 'nombre': 'Lyton Levison Mphande', 'altura': 127.0},
      'mas_ligero': {'id': 21049, 'nombre': 'Choi Myong-Hui', 'peso': 25.0},
      'mas_pesado': {'id': 12177, 'nombre': 'Ricardo Blas, Jr.', 'peso': 214.0}}

```

1.4 3. Cantidad de medallas y rendimiento

1.4.1 Análisis

A continuación se obtendrá un mapa de calor de país $\times$ deporte (en el que la intensidad sea el número total de medallas). Para favorecer la claridad se mostrarán únicamente los 30 países con más medallas

```

[14]: # Descartamos filas donde no se ha ganado ninguna medalla
medallas = athlete_medals_df.dropna(subset=["Medal"])

# Recuento de medallas por deporte para cada país
tabla = pd.crosstab(medallas["NOC"], medallas["Sport"])
tabla = tabla.loc[
    tabla.sum(axis=1).sort_values(ascending=False).index,
    tabla.sum(axis=0).sort_values(ascending=False).index
].head(30)

fig, ax = plt.subplots(
    figsize=(
        max(30, len(tabla.columns) * 0.65),
        max(12, len(tabla) * 0.45)
    )
)

sns.heatmap(
    tabla,

```

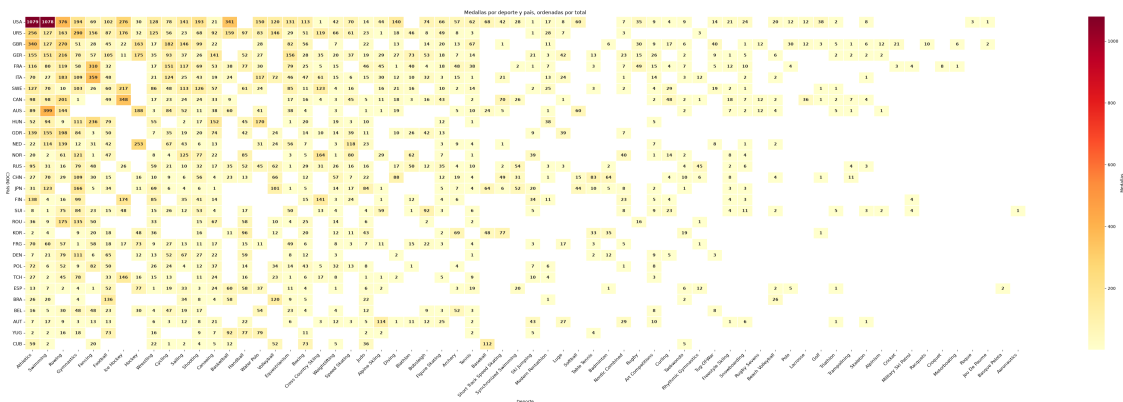
```

mask=tabla.eq(0),
annot=True,
fmt="d",
annot_kws={"fontsize": 10, "fontweight": "bold"},
cmap="YlOrRd",
linewidths=0.3,
linecolor="white",
xticklabels=True,
yticklabels=True,
cbar_kws={"label": "Medallas"},
ax=ax
)

ax.set_xticklabels(
    ax.get_xticklabels(), rotation=45, ha="right", fontsize=11
)
ax.set_yticklabels(
    ax.get_yticklabels(), rotation=0, fontsize=11
)
ax.set_xlabel("Deporte")
ax.set_ylabel("País (NOC)")
ax.set_title("Medallas por deporte y país, ordenadas por total")

plt.tight_layout()
plt.show()

```



Ahora el ranquin de los 10 países con mayor número total de medallas (de cualquier tipo) y, para cada uno de esos 10 países, el deporte en el que más medallas ha conseguido

```

[15]: # Descartamos filas donde no se ha ganado ninguna medalla
medallas = athlete_medals_df.dropna(subset=["Medal"])

```

```

# Recuento de medallas por deporte para cada país
tabla = pd.crosstab(medallas["NOC"], medallas["Sport"])
tabla = tabla.loc[
    tabla.sum(axis=1).sort_values(ascending=False).index,
    tabla.sum(axis=0).sort_values(ascending=False).index
].head(10)

tabla["Deporte_mas_laureado"] = tabla.idxmax(axis=1)
tabla["Total_medallas"] = tabla.drop(
    columns="Deporte_mas_laureado"
).sum(axis=1)

display(tabla[["Total_medallas", "Deporte_mas_laureado"]])

```

Sport	Total_medallas	Deporte_mas_laureado
NOC		
USA	5633	Athletics
URS	2660	Gymnastics
GBR	2075	Athletics
GER	2049	Rowing
FRA	1773	Fencing
ITA	1627	Fencing
SWE	1535	Ice Hockey
CAN	1347	Ice Hockey
AUS	1314	Swimming
HUN	1131	Fencing

Ahora se mostrarán los 5 países líderes en medallas, y su evolución por décadas de su cantidad de medallas.

```

[16]: medallas = athlete_medals_df.dropna(subset=["Medal"]).copy()

# Cinco países/NOC con más medallas en todo el periodo.
lideres = medallas["NOC"].value_counts().head(5).index

# Cada década se identifica por su año inicial.
medallas["Decada"] = (medallas["Year"] // 10) * 10

evolucion = pd.crosstab(
    medallas["Decada"],
    medallas["NOC"]
).reindex(columns=lideres, fill_value=0)

# Incluimos todas las décadas del periodo estudiado.
decadas = range(
    int(medallas["Decada"].min()),
    int(medallas["Decada"].max()) + 10,
    10

```

```

)
evolucion = evolucion.cumsum(axis=0)

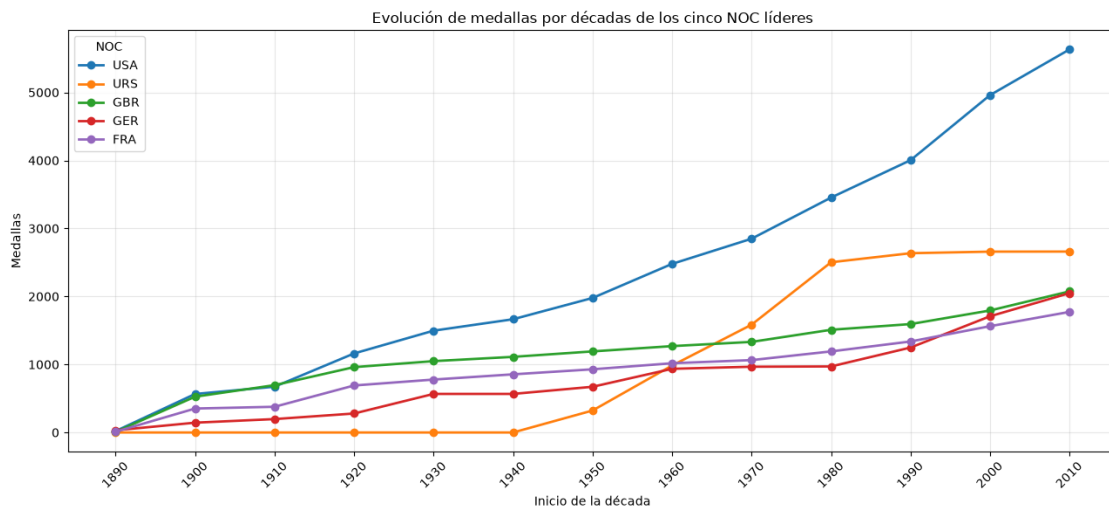
ax = evolucion.plot(
    figsize=(13, 6),
    marker="o",
    linewidth=2
)

ax.set_xlabel("Inicio de la década")
ax.set_ylabel("Medallas")
ax.set_title("Evolución de medallas por décadas de los cinco NOC líderes")
ax.set_xticks(evolucion.index)
ax.tick_params(axis="x", rotation=45)
ax.legend(title="NOC")
ax.grid(alpha=0.3)

plt.tight_layout()
plt.show()

display(evolucion)

```



NOC	USA	URS	GBR	GER	FRA
Decada					
1890	20	0	9	32	11
1900	566	0	526	144	352
1910	673	0	696	197	378
1920	1162	0	961	279	691
1930	1497	0	1049	567	778
1940	1665	0	1112	567	855

1950	1978	323	1192	672	929
1960	2479	984	1270	936	1018
1970	2849	1585	1332	967	1064
1980	3454	2503	1510	971	1192
1990	4004	2636	1594	1249	1338
2000	4961	2659	1794	1709	1563
2010	5633	2660	2075	2049	1773

A continuación se elige **la gimnasia** para analizar si hay relación entre la altura/peso de los deportistas y probabilidad de medalla en función del país. Tras ello también compararemos todos los países para el deporte elegido. Por claridad en los resultados separaremos las distribuciones de nadadores entre hombre y mujer.

```
[ ]: deporte = "Gymnastics"

# -----
# FILTRAMOS POR SEXO
# -----

deporte_males = athlete_medals_df.loc[
    (athlete_medals_df["Sport"] == deporte)
    & (athlete_medals_df["Sex"] == "M")
]

deporte_females = athlete_medals_df.loc[
    (athlete_medals_df["Sport"] == deporte)
    & (athlete_medals_df["Sex"] == "F")
]

# -----
# AGRUPAMOS POR DEPORTISTA
# -----

deporte_males = (
    deporte_males
    .groupby("ID", as_index=False)
    .agg(
        Name=("Name", "first"),
        NOC=("NOC", "first"),
        Altura=("Height", "first"),
        Peso=("Weight", "first"),
        Eventos=("ID", "size"),
        Numero_medallas=("Medal", "count")
    )
    .dropna(subset=["Altura", "Peso"]).sort_values(
        "Numero_medallas", ascending=False
    )
)
```

```

deporte_females = (
    deporte_females
    .groupby("ID", as_index=False)
    .agg(
        Name=("Name", "first"),
        NOC=("NOC", "first"),
        Altura=("Height", "first"),
        Peso=("Weight", "first"),
        Eventos=("ID", "size"),
        Numero_medallas=("Medal", "count")
    )
).dropna(subset=["Altura", "Peso"]).sort_values(
    "Numero_medallas", ascending=False
)

# Ratio individual de medallas
deporte_males["laureatus_rating"] = (
    deporte_males["Numero_medallas"] / deporte_males["Eventos"]
)

deporte_females["laureatus_rating"] = (
    deporte_females["Numero_medallas"] / deporte_females["Eventos"]
)

# =====
# HOMBRES
# =====

display(deporte_males)

# Agrupamos por país
países_hombres = (
    deporte_males
    .groupby("NOC", as_index=False)
    .agg(
        Altura=("Altura", "mean"),
        Peso=("Peso", "mean"),
        Eventos=("Eventos", "sum"),
        Medallas=("Numero_medallas", "sum"),
        Nadadores=("ID", "count")
    )
)

# Probabilidad de medalla
países_hombres["Prob_medalla"] = (
    países_hombres["Medallas"] / países_hombres["Eventos"]
)

```

```

# SCATTERPLOT HOMBRES
plt.figure(figsize=(14, 9))

sns.scatterplot(
    data=países_hombres,
    x="Altura",
    y="Peso",
    hue="Prob_medalla",
    size="Nadadores",
    sizes=(30, 400),
    palette="YlOrRd"
)

for _, fila in países_hombres.iterrows():
    plt.text(
        fila["Altura"],
        fila["Peso"],
        fila["NOC"],
        fontsize=7
    )

plt.xlabel("Altura media (cm)")
plt.ylabel("Peso medio (kg)")
plt.title(
    f"{deporte} masculino: altura, peso y probabilidad de medalla por país"
)
plt.tight_layout()
plt.show()

# HEATMAP HOMBRES
datos = deporte_males.copy()

limites_altura = np.arange(
    np.floor(datos["Altura"].min()),
    np.floor(datos["Altura"].max()) + 3,
    3
)

limites_peso = np.arange(
    np.floor(datos["Peso"].min()),
    np.floor(datos["Peso"].max()) + 3,
    3
)

```

```

datos["Grupo_altura"] = pd.cut(
    datos["Altura"],
    bins=limites_altura,
    right=False
)

datos["Grupo_peso"] = pd.cut(
    datos["Peso"],
    bins=limites_peso,
    right=False
)

tabla = datos.pivot_table(
    index="Grupo_altura",
    columns="Grupo_peso",
    values="Numero_medallas",
    aggfunc="sum",
    observed=False,
    fill_value=0
)

tabla.index = tabla.index.categories.left.astype(int)
tabla.columns = tabla.columns.categories.left.astype(int)

fig, ax = plt.subplots(figsize=(22, 16))

sns.heatmap(
    tabla,
    mask=tabla.eq(0),
    annot=True,
    fmt=".0f",
    annot_kws={"fontsize": 8},
    cmap="YlOrRd",
    cbar_kws={"label": "Total de medallas"},
    ax=ax
)

ax.invert_yaxis()
ax.set_xlabel("Peso (kg): intervalo de 3 kg")
ax.set_ylabel("Altura (cm): intervalo de 3 cm")
ax.set_title(
    f"Medallas de hombres en {deporte} por altura y peso"
)

plt.xticks(rotation=90)
plt.yticks(rotation=0)
plt.tight_layout()

```

```

plt.show()

# =====
# MUJERES
# =====

display(deporte_females)

# Agrupamos por país
países_mujeres = (
    deporte_females
    .groupby("NOC", as_index=False)
    .agg(
        Altura=("Altura", "mean"),
        Peso=("Peso", "mean"),
        Eventos=("Eventos", "sum"),
        Medallas=("Numero_medallas", "sum"),
        Nadadoras=("ID", "count")
    )
)

# Probabilidad de medalla
países_mujeres["Prob_medalla"] = (
    países_mujeres["Medallas"] / países_mujeres["Eventos"]
)

# SCATTERPLOT MUJERES
plt.figure(figsize=(14, 9))

sns.scatterplot(
    data=países_mujeres,
    x="Altura",
    y="Peso",
    hue="Prob_medalla",
    size="Nadadoras",
    sizes=(30, 400),
    palette="YlOrRd"
)

for _, fila in países_mujeres.iterrows():
    plt.text(
        fila["Altura"],
        fila["Peso"],
        fila["NOC"],
        fontsize=7
    )

```

```

plt.xlabel("Altura media (cm)")
plt.ylabel("Peso medio (kg)")
plt.title(
    f"{deporte} femenino: altura, peso y probabilidad de medalla por país"
)
plt.tight_layout()
plt.show()

# HEATMAP MUJERES
datos = deporte_females.copy()

limites_altura = np.arange(
    np.floor(datos["Altura"].min()),
    np.floor(datos["Altura"].max()) + 3,
    3
)

limites_peso = np.arange(
    np.floor(datos["Peso"].min()),
    np.floor(datos["Peso"].max()) + 3,
    3
)

datos["Grupo_altura"] = pd.cut(
    datos["Altura"],
    bins=limites_altura,
    right=False
)

datos["Grupo_peso"] = pd.cut(
    datos["Peso"],
    bins=limites_peso,
    right=False
)

tabla = datos.pivot_table(
    index="Grupo_altura",
    columns="Grupo_peso",
    values="Numero_medallas",
    aggfunc="sum",
    observed=False,
    fill_value=0
)

tabla.index = tabla.index.categories.left.astype(int)

```

```

tabla.columns = tabla.columns.categories.left.astype(int)

fig, ax = plt.subplots(figsize=(22, 16))

sns.heatmap(
    tabla,
    mask=tabla.eq(0),
    annot=True,
    fmt=".0f",
    annot_kws={"fontsize": 8},
    cmap="YlOrRd",
    cbar_kws={"label": "Total de medallas"},
    ax=ax
)

ax.invert_yaxis()
ax.set_xlabel("Peso (kg): intervalo de 3 kg")
ax.set_ylabel("Altura (cm): intervalo de 3 cm")
ax.set_title(
    f"Medallas de mujeres en {deporte} por altura y peso"
)

plt.xticks(rotation=90)
plt.yticks(rotation=0)
plt.tight_layout()
plt.show()

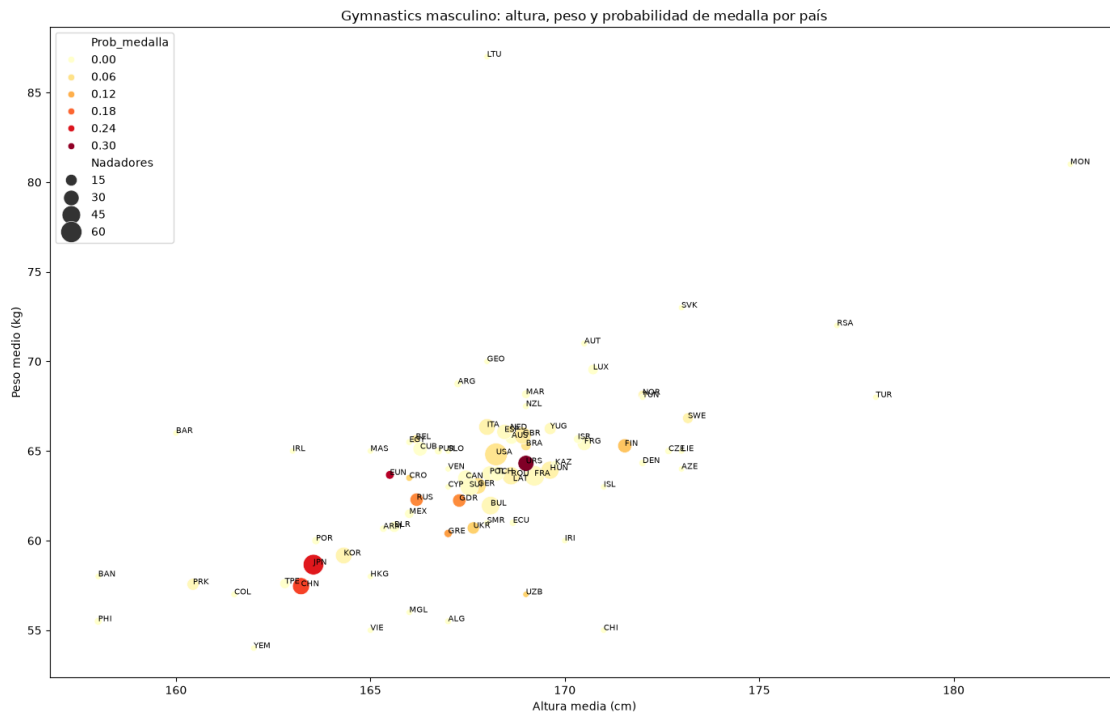
```

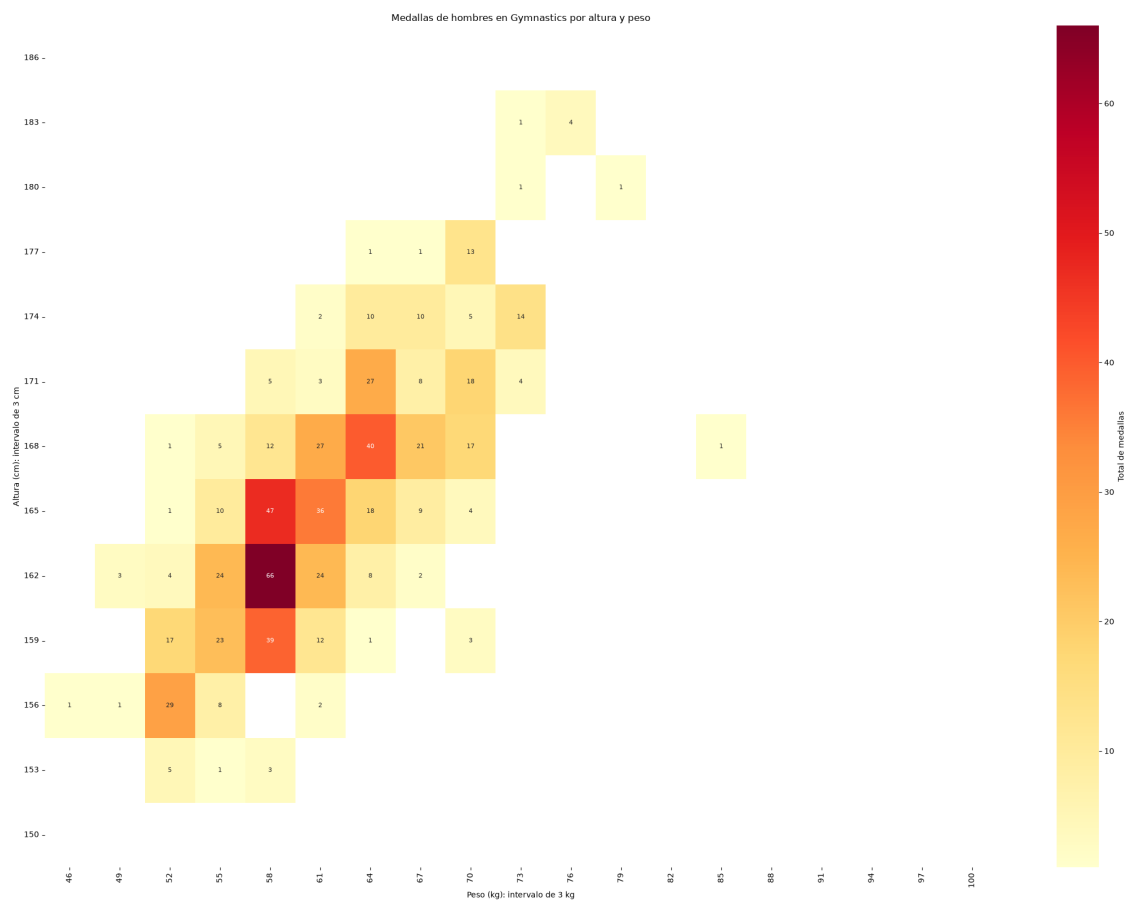
	ID	Name	NOC	Altura	Peso	Eventos	\
64	4198	Nikolay Yefimovich Andrianov	URS	166.0	60.0	24	
1716	89187	Takashi Ono	JPN	160.0	58.0	32	
2093	109161	Borys Anfiyanovych Shakhlin	URS	171.0	71.0	24	
1100	57998	Sawao Kato	JPN	163.0	59.0	24	
1644	85286	Aleksey Yuryevich Nemov	RUS	174.0	75.0	21	
...	...	...	...	...	...	...	
1023	54248	Alexander "Sasha" Jeltkov	CAN	178.0	78.0	11	
1030	54442	Sidney "Sid" Jensen	CAN	168.0	63.0	8	
1033	54559	Jeong Jin-Su	KOR	160.0	55.0	24	
1034	54574	Jeong Ri-Gwang	KOR	162.0	58.0	8	
990	52801	Ivan Yanev Ivanov	BUL	158.0	56.0	7	

	Numero_medallas	laureatus_rating
64	15	0.625000
1716	13	0.406250
2093	13	0.541667
1100	12	0.500000
1644	12	0.571429
...	...	...
1023	0	0.000000

1030	0	0.000000
1033	0	0.000000
1034	0	0.000000
990	0	0.000000

[1042 rows x 8 columns]



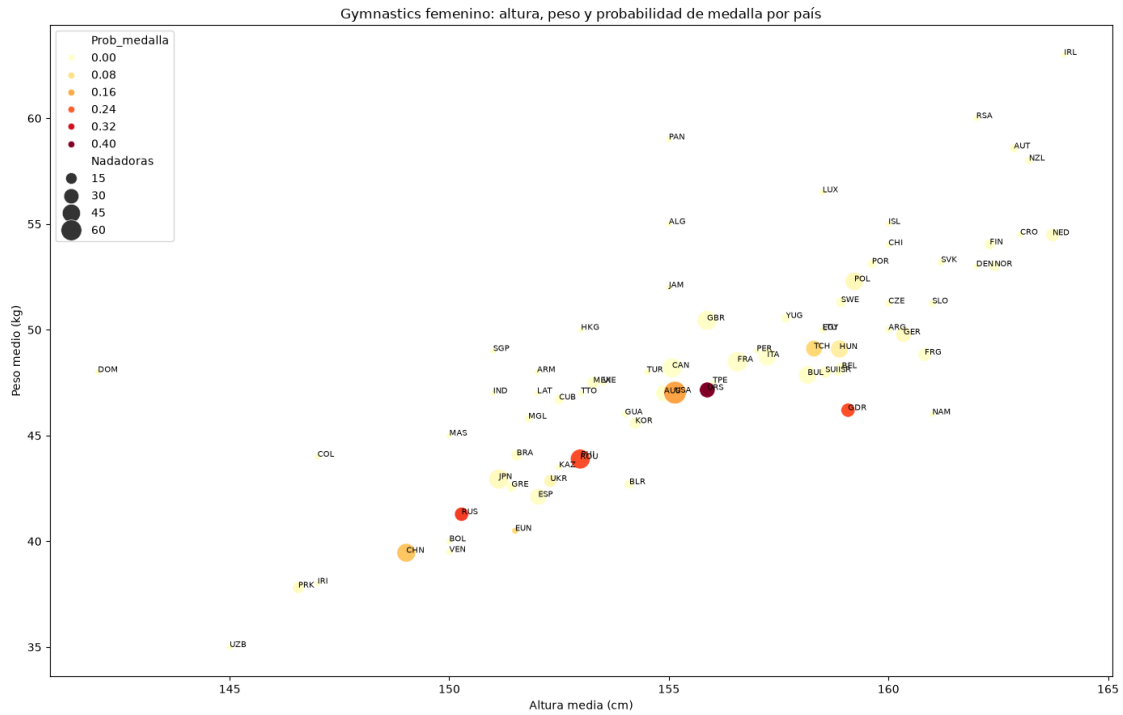


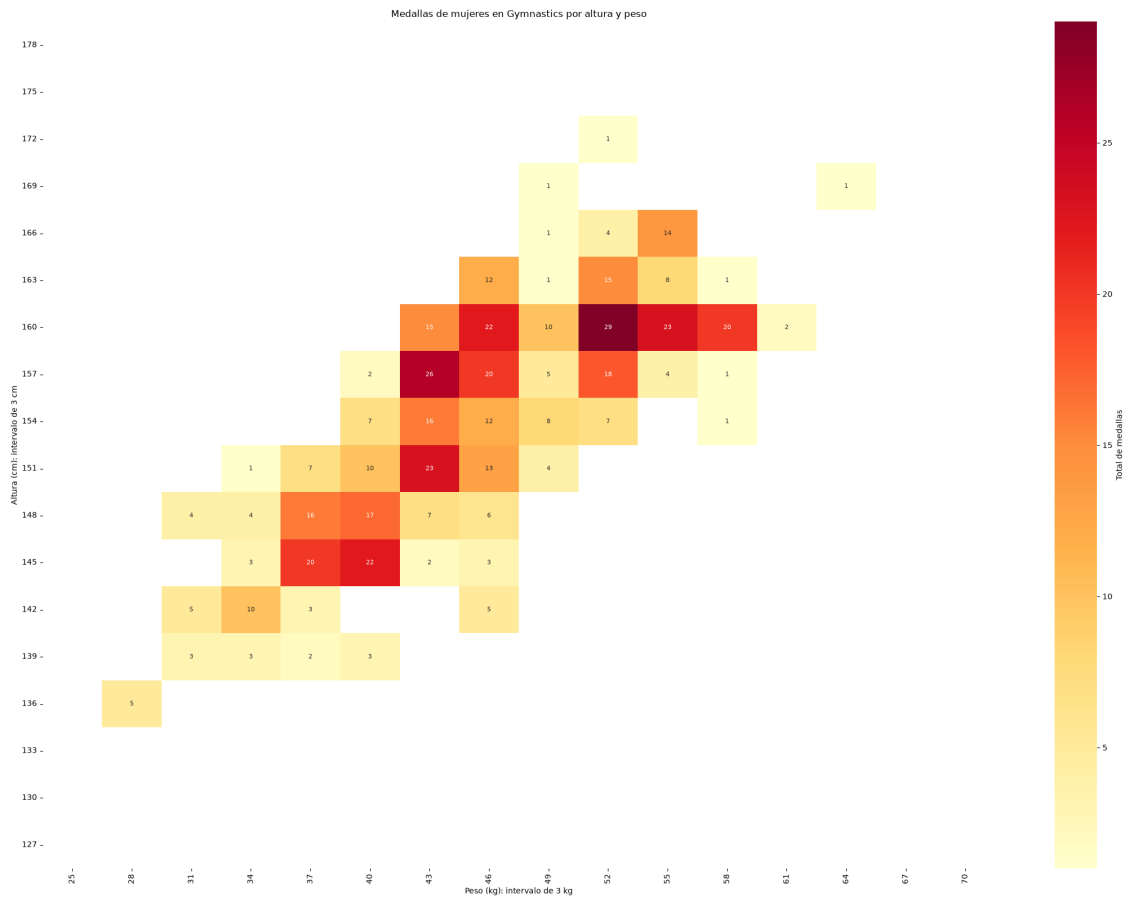
	ID	Name	NOC	Altura	Peso	\
691	67046	Larysa Semenivna Latynina (Diriy-)	URS	161.0	52.0	
176	18826	Vra slavsk (-Odloilov)	TCH	160.0	58.0	
39	5700	Polina Hryhorivna Astakhova	URS	166.0	56.0	
216	22570	Nadia Elena Conneci (-Conner)	ROU	162.0	45.0	
1336	122726	Lyudmila Ivanovna Turishcheva (-Borzova)	URS	160.0	52.0	
...	...	...	...	...	...	
594	58603	Karen Barbara Kelsall	CAN	150.0	37.0	
595	58720	Karen Kennedy	GBR	162.0	50.0	
596	58821	Angelika Kern	FRG	158.0	46.0	
600	58934	Rose-Marie "Romi" Kessler	SUI	159.0	48.0	
571	56612	Ariella Kaeslin	SUI	165.0	55.0	

	Eventos	Numero_medallas	laureatus_rating
691	19	18	0.947368
176	18	11	0.611111
39	19	10	0.526316
216	12	9	0.750000
1336	18	9	0.500000
...	...	...	...

594	6	0	0.000000
595	5	0	0.000000
596	12	0	0.000000
600	6	0	0.000000
571	5	0	0.000000

[1130 rows x 8 columns]





En los hombres, la mayoría de las medallas se concentra aproximadamente entre 159–168 cm de altura y 55–67 kg de peso. El scatterplot masculino está sesgado, y se concentra en el cuadrante inferior izquierdo de la gráfica. Esto quiere decir que la gimnasia favorece las estaturas bajas y los pesos ligeros. El heatmap también permite ver que un peso ligero influye más en la probabilidad de medalla que una altura baja.

En las mujeres, las medallas se concentran principalmente entre 145–160 cm y 37–55 kg. Ellas presentan una distribución más equilibrada que los hombres, con valores un poco más dispersos. De media, y como en otros deportes, presentan una altura y peso inferiores a los de los hombres.

`ranking_paises_medallas(df_medallas, k)`

Devuelve devuelva el Top-k de países, siguiendo una serie de reglas de desempate: 1. El país con másoros 2. Si empatan, el de más platas 3. Si empatan, el de más bronces 4. Si empatan, el del primer año en que el país logró una medalla de cualquier tipo (más antiguo gana) 5. Si persiste el empate, por orden alfabético del país

```
[55]: def ranking_paises_medallas(df, k):
      response = []
      datos = (
          df.groupby("NOC", as_index=False)
```

```

.agg(
    Golds=("Medal", lambda s: (s == "Gold").sum()),
    Silvers=("Medal", lambda s: (s == "Silver").sum()),
    Bronzes=("Medal", lambda s: (s == "Bronze").sum()),
    Eventos=("ID", "size"),
    Numero_medallas=("Medal", "count"),
    Primera_participacion=("Year", "min")
)
.sort_values("Numero_medallas", ascending=False)
.reset_index(drop=True)
).sort_values(
    ["Golds", "Silvers", "Bronzes", "Primera_participacion", "NOC"],
    ascending=[False, False, False, True, True]
).head(k)

for index, (_, row) in enumerate(datos.iterrows()):
    response.append({
        "orden": index + 1,
        "país": row["NOC"],
        "oro": row["Golds"],
        "plata": row["Silvers"],
        "bronce": row["Bronzes"],
        "primer_año": row["Primera_participacion"],
    })

for item in response:
    print(item)

```

```
ranking_paises_medallas(athlete_medals_df, 10)
```

```

{'orden': 1, 'país': 'USA', 'oro': 2637, 'plata': 1642, 'bronce': 1354,
'primer_año': 1896}
{'orden': 2, 'país': 'URS', 'oro': 1137, 'plata': 783, 'bronce': 740,
'primer_año': 1952}
{'orden': 3, 'país': 'GER', 'oro': 690, 'plata': 647, 'bronce': 712,
'primer_año': 1896}
{'orden': 4, 'país': 'GBR', 'oro': 679, 'plata': 743, 'bronce': 653,
'primer_año': 1896}
{'orden': 5, 'país': 'ITA', 'oro': 574, 'plata': 525, 'bronce': 528,
'primer_año': 1896}
{'orden': 6, 'país': 'FRA', 'oro': 500, 'plata': 607, 'bronce': 666,
'primer_año': 1896}
{'orden': 7, 'país': 'SWE', 'oro': 478, 'plata': 522, 'bronce': 535,
'primer_año': 1896}
{'orden': 8, 'país': 'CAN', 'oro': 462, 'plata': 435, 'bronce': 450,
'primer_año': 1900}
{'orden': 9, 'país': 'GDR', 'oro': 437, 'plata': 345, 'bronce': 302,

```

```
'primer_año': 1968}  
{'orden': 10, 'país': 'HUN', 'oro': 429, 'plata': 331, 'bronce': 371,  
'primer_año': 1896}
```